

Réalisation d'un jeu interactif en Java

1. Introduction

L'objectif de ce projet est de créer un petit jeu très simple avec une fenêtre à l'écran dans laquelle un cercle est affiché. Ce cercle représente un objet qui peut monter et descendre. Le but principal n'est pas de créer un jeu compliqué, mais de comprendre comment un programme peut afficher quelque chose à l'écran, réagir aux actions de l'utilisateur et fonctionner de manière fluide.

Dans cette application, le cercle descend tout seul au fil du temps. Lorsque l'utilisateur clique avec la souris dans la fenêtre, le cercle monte vers le haut, comme s'il sautait. Cela permet de montrer comment un programme peut réagir aux actions de l'utilisateur et mettre à jour l'affichage en conséquence.

Le projet a aussi pour objectif d'apprendre à organiser correctement le travail. Chaque partie du programme a un rôle bien précis : une partie s'occupe de mémoriser les informations importantes, une autre s'occupe de l'affichage, et une autre encore gère les actions de l'utilisateur. Cette organisation rend le programme plus clair et plus facile à comprendre.

Ce travail constitue une première étape avant des projets plus complets. Il permet de poser les bases nécessaires pour ajouter plus tard de nouvelles fonctionnalités, comme un parcours à suivre ou des déplacements plus complexes.

2. Analyse globale

Analyse globale des fonctionnalités

Le projet a pour objectif de développer une application graphique interactive en Java, structurée selon le patron de conception MVC. Afin d'atteindre cet objectif, plusieurs fonctionnalités principales ont été identifiées. Ces fonctionnalités constituent la base du projet et seront détaillées et planifiées dans la section suivante (plan de développement).

Les fonctionnalités principales du projet sont les suivantes :

1. Affichage d'une interface graphique
2. Interaction utilisateur par clic de souris
3. Gestion du mouvement vertical du cercle
4. Mise à jour automatique de l'affichage

5. Génération et affichage d'un parcours sous forme de ligne brisée
6. Organisation du code selon le modèle MVC

Analyse détaillée des fonctionnalités

1. Affichage de l'interface graphique

Cette fonctionnalité consiste à créer une fenêtre graphique et à y afficher les éléments visuels du projet, en particulier le cercle représentant l'objet principal.

- **Difficulté** : faible
- **Priorité** : très élevée

Cette fonctionnalité est indispensable, car aucune autre fonctionnalité ne peut être utilisée sans interface graphique visible.

2. Interaction utilisateur par clic de souris

Lorsqu'un clic est détecté, une action est déclenchée afin de modifier l'état du modèle.

- **Difficulté** : moyenne
- **Priorité** : très élevée

Elle est essentielle pour rendre l'application interactive et permet de faire le lien entre l'utilisateur et le comportement du cercle.

3. Gestion du mouvement vertical du cercle

Le cercle doit monter lorsqu'un clic est effectué et redescendre progressivement lorsqu'aucune interaction n'a lieu. Cette fonctionnalité repose sur la modification de la position verticale du cercle stockée dans le modèle.

- **Difficulté** : moyenne
- **Priorité** : élevée

4. Mise à jour automatique de l'affichage

Afin de garantir une animation fluide, l'affichage doit être rafraîchi régulièrement. Cette fonctionnalité est assurée par un thread dédié qui redessine l'interface à intervalles réguliers.

- **Difficulté** : moyenne
- **Priorité** : élevée

5. Génération et affichage d'un parcours sous forme de ligne brisée

Cette fonctionnalité consiste à générer une ligne brisée à partir d'une liste de points calculés automatiquement, en respectant des contraintes de position. Cette ligne est ensuite affichée dans la vue.

- **Difficulté** : élevée
- **Priorité** : moyenne

6. Organisation du code selon le modèle MVC

Le projet est structuré selon le patron de conception MVC afin de séparer clairement les responsabilités :

- Le modèle gère les données et la logique,
- La vue gère l'affichage,
- Le contrôleur gère les interactions utilisateur.
- **Difficulté** : moyenne
- **Priorité** : très élevée

Cette organisation améliore la lisibilité, la maintenabilité et l'évolutivité du projet.

3. Plan de développement

3.1. GANTT



Le diagramme de Gantt présente l'organisation temporelle du projet.

Les périodes notées S1 à S7 correspondent aux différentes séances de travail.

Chaque tâche correspond à une fonctionnalité ou sous-fonctionnalité décrite dans l'analyse détaillée.

3.2. Utilisation d'un assistant de programmation (GitHub Copilot)

Au cours du projet, nous avons utilisé GitHub Copilot comme outil d'assistance à la programmation.

Copilot a été utilisé principalement lors des phases de développement afin de proposer des squelettes de code à partir de commentaires écrits en langage naturel.

Les propositions générées n'ont jamais été utilisées telles quelles : elles ont systématiquement été relues, corrigées et adaptées aux contraintes du projet).

Cet outil a permis de gagner du temps sur les parties répétitives du code, tout en conservant une compréhension complète du fonctionnement du programme.

4. Choix de conception et organisation du projet (MVC)

Le modèle

Le **modèle** regroupe les classes responsables de l'état du jeu et de son évolution. Il contient notamment les informations suivantes :

- La hauteur du cercle ;
- L'avancement horizontal du jeu ;
- La structure du parcours sous forme de points ;
- L'état de collision entre le cercle et la ligne.

Les mises à jour du modèle sont effectuées par différents threads, ce qui permet de gérer :

- La gravité (descente automatique du cercle) ;
- Le défilement horizontal du jeu ;
- La détection de collision.

La vue

La **vue** est chargée de représenter graphiquement l'état du jeu.

Elle lit les informations contenues dans le modèle et les transforme en éléments visuels:

- Dessin du cercle
- Dessin du parcours

- Animation visuelle en cas de collision.

Le contrôleur

Le **contrôleur** gère les interactions de l'utilisateur avec l'interface graphique.

Il intercepte les événements de clic de souris et transmet les actions correspondantes au modèle.

Cette approche permet de :

- Centraliser la gestion des interactions utilisateur ;
- Éviter toute modification directe du modèle depuis la vue ;
- Respecter le principe de séparation des responsabilités.

Utilisation des threads

Plusieurs threads sont utilisés afin de gérer les différentes dynamiques du jeu de manière indépendante :

- Un thread pour la descente automatique du cercle (gravité) ;
- Un thread pour le défilement horizontal du jeu ;
- Un thread pour le rafraîchissement de l'affichage ;
- Un thread pour la détection des collisions.

L'utilisation de threads permet d'obtenir une animation fluide et de dissocier les différentes tâches du programme.

Intérêt du choix MVC

Le choix de l'architecture MVC présente plusieurs avantages :

- Une meilleure lisibilité du code ;
- Une séparation claire des responsabilités ;
- Une facilité d'évolution et de maintenance ;
- Une meilleure cohérence avec les principes enseignés dans le cours.

Ce choix de conception a permis de structurer efficacement le projet et de répondre aux exigences fonctionnelles tout en respectant les bonnes pratiques de programmation.

5. Conception détaillée

1. Affichage de l'interface graphique

Objectif

Cette fonctionnalité a pour objectif de fournir une interface graphique permettant à l'utilisateur de visualiser le jeu et ses différents éléments.

1.a Création de la fenêtre graphique

Une fenêtre graphique est créée à l'aide de la bibliothèque Swing. Cette fenêtre constitue le support principal de l'application. Elle définit les dimensions de l'espace de jeu et permet d'accueillir les éléments graphiques.

1.b Création d'une zone de dessin

À l'intérieur de la fenêtre, une zone de dessin est mise en place à l'aide d'un composant graphique. Cette zone est dédiée à l'affichage des éléments du jeu. Elle permet de centraliser le dessin du cercle et du parcours, et sert de lien entre le modèle et l'affichage.

1.c Dessin du cercle

Le cercle représente l'objet principal du jeu. Il est dessiné à partir des données contenues dans le modèle, notamment sa hauteur. Le cercle reste fixe horizontalement à l'écran, ce qui donne l'impression que le décor se déplace autour de lui. Sa position verticale dépend directement de la valeur stockée dans le modèle.

1.d Dessin de la ligne brisée

La ligne brisée représente le parcours du jeu. Elle est constituée d'une succession de segments reliant des points générés dans le modèle. Lors de l'affichage, les coordonnées des points sont transformées afin de s'adapter au repère graphique de la fenêtre.

2. Interaction utilisateur par clic de souris

Objectif

Permettre à l'utilisateur d'interagir avec le jeu de manière simple et intuitive à l'aide de la souris.

2.a Détection du clic souris

Les clics de souris sont capturés au niveau de la zone graphique. Le programme est capable de détecter lorsqu'un utilisateur clique dans la fenêtre du jeu.

2.b Transmission de l'événement au modèle

Lorsqu'un clic est détecté, l'événement est transmis au modèle par l'intermédiaire d'un contrôleur. Cette étape permet de respecter l'architecture MVC en évitant toute modification directe de l'affichage.

2.c Déclenchement du saut du cercle

Le clic provoque une modification de la hauteur du cercle dans le modèle. Cette modification simule un saut, ce qui permet à l'utilisateur d'influencer le comportement du cercle pendant le jeu.

3. Gestion du mouvement vertical du cercle

Objectif

Gérer le déplacement vertical du cercle de manière réaliste et contrôlée.

3.a Stockage de la hauteur du cercle

La hauteur du cercle est stockée dans le modèle sous la forme d'une variable numérique. Cette valeur représente la position verticale du cercle dans le repère logique du jeu.

3.b Limitation de la hauteur maximale

Une limite maximale est définie afin d'empêcher le cercle de sortir de la zone visible par le haut. Cela garantit la cohérence de l'affichage.

3.c Limitation de la hauteur minimale

De la même manière, une limite minimale est définie pour éviter que le cercle ne descende trop bas et disparaisse de la zone de jeu.

3.d Descente automatique du cercle

Un thread dédié est utilisé pour simuler la gravité. En l'absence de clic utilisateur, ce thread diminue progressivement la hauteur du cercle, ce qui donne l'impression qu'il retombe naturellement.

4. Mise à jour automatique de l'affichage

Objectif

Assurer une animation fluide et continue du jeu.

4.a Rafraîchissement périodique de l'affichage

Un thread spécifique est chargé de redessiner régulièrement l'interface graphique. Il demande à Swing de rafraîchir l'affichage à intervalles réguliers.

4.b Indépendance entre logique et affichage

Le rafraîchissement graphique est totalement indépendant de la logique du jeu. Cette séparation permet une meilleure organisation du code et respecte le modèle MVC.

5. Génération et affichage du parcours (ligne brisée)

Objectif

Créer un parcours dynamique servant de décor au jeu.

5.a Génération des points du parcours

Le parcours est généré automatiquement à partir d'une suite de points. Les positions horizontales sont choisies aléatoirement dans un intervalle défini, tandis que les hauteurs respectent des limites verticales.

5.b Respect des limites verticales du parcours

Les hauteurs des points sont contraintes afin que le parcours reste visible et cohérent avec la zone de jeu.

5.c Affichage du parcours à partir des points

Les points générés sont reliés entre eux par des segments afin de former une ligne brisée visible à l'écran.

6. Défilement horizontal du jeu

Objectif

Simuler un déplacement horizontal continu du jeu.

6.a Stockage de l'avancement horizontal

L'avancement horizontal est mémorisé dans le modèle. Cette valeur représente la progression du jeu dans le temps.

6.b Mise à jour régulière de l'avancement

Un thread est chargé d'incrémenter régulièrement l'avancement. Cela permet de faire évoluer le jeu de manière continue.

6.c Décalage du parcours à l'affichage

Lors de l'affichage, les coordonnées du parcours sont décalées en fonction de l'avancement, ce qui donne l'illusion que le décor se déplace.

7. Détection de collision avec le parcours (nouvelle fonctionnalité)

Objectif

Détecter les collisions entre le cercle et la ligne brisée et déclencher une animation visuelle.

7.a Calcul de la hauteur de la ligne sous le cercle

Pour détecter une collision, le programme calcule la hauteur du parcours à l'abscisse du cercle à l'aide de la pente du segment concerné.

7.b Comparaison de la position du cercle avec le parcours

La hauteur du cercle est comparée à celle du parcours afin de déterminer s'il y a contact entre les deux éléments.

7.c Détection automatique de la collision

Un thread dédié effectue régulièrement cette vérification afin de détecter les collisions en temps réel.

7.d Déclenchement d'une animation visuelle

En cas de collision, une animation visuelle est déclenchée, par exemple un changement temporaire de couleur du cercle, afin de signaler l'événement à l'utilisateur.

6. Documentation utilisateur

L'application se lance en exécutant la classe principale du projet. Une fenêtre graphique s'ouvre alors automatiquement.

La fenêtre affiche un cercle représentant l'objet principal du jeu ainsi qu'une ligne brisée représentant le parcours. Le cercle reste fixe horizontalement à l'écran tandis que le parcours défile de manière continue, ce qui donne l'illusion d'un déplacement horizontal.

L'utilisateur peut interagir avec l'application en cliquant dans la fenêtre. Chaque clic provoque un saut du cercle vers le haut. En l'absence de clic, le cercle redescend automatiquement sous l'effet de la gravité simulée par le programme.

Le parcours est généré de manière dynamique et s'étend de façon infinie. Lorsque le cercle entre en collision avec la ligne brisée, une animation visuelle est déclenchée afin de signaler la collision à l'utilisateur.

L'application ne nécessite aucune configuration particulière et peut être fermée en utilisant le bouton de fermeture de la fenêtre.

7. Documentation développeur

Le projet est structuré selon l'architecture MVC (Modèle – Vue – Contrôleur), afin de séparer clairement les responsabilités et de faciliter la maintenance et l'évolution du code.

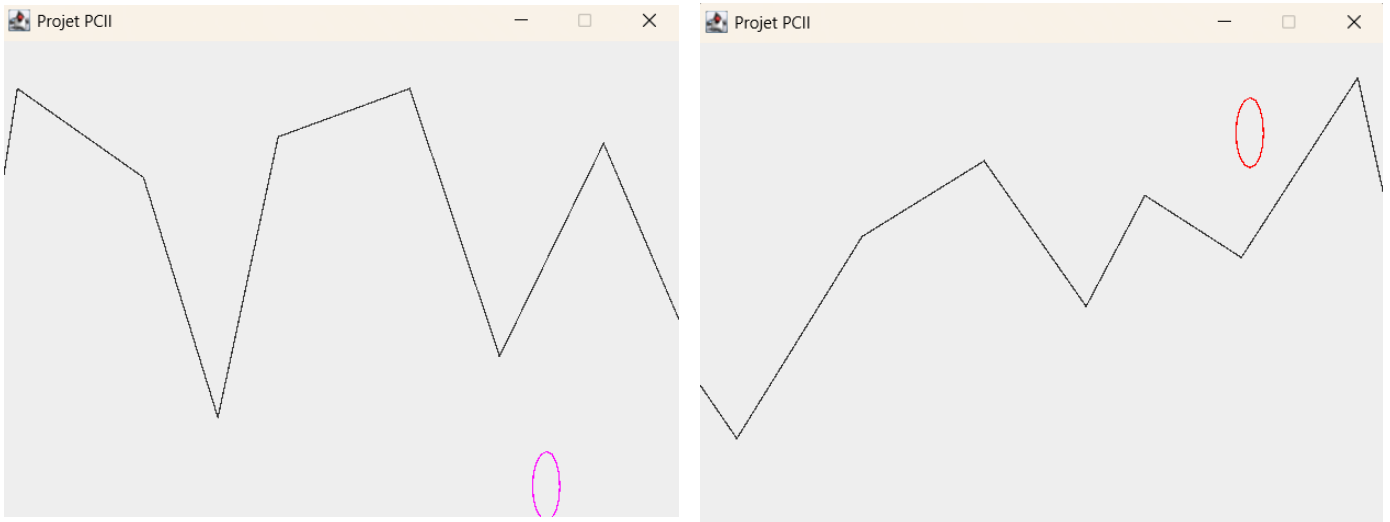
Le modèle regroupe les classes responsables de l'état du jeu et de sa logique. Il contient notamment la position du cercle, l'avancement horizontal et la structure du parcours sous forme de points. Les comportements automatiques du jeu sont gérés par plusieurs threads indépendants.

La vue est chargée de l'affichage graphique. Elle lit les informations contenues dans le modèle et les transforme en éléments visuels à l'aide de la bibliothèque Swing. La vue ne modifie jamais directement les données du modèle.

Le contrôleur gère les interactions utilisateur, en particulier les clics de souris. Il transmet les actions correspondantes au modèle, ce qui permet de respecter l'architecture MVC.

Pour faire évoluer le projet, il est possible d'ajouter de nouvelles fonctionnalités en créant de nouveaux threads ou de nouveaux éléments graphiques, tout en conservant la séparation entre modèle, vue et contrôleur.

8. Résultat



Le programme final fonctionne conformément aux objectifs définis dans le cahier des charges. L'interface graphique s'affiche correctement et le cercle réagit aux interactions de l'utilisateur en temps réel.

Le parcours est généré dynamiquement et défile de manière continue. La collision entre le cercle et la ligne brisée est correctement détectée et déclenche une animation visuelle, comme illustré par les captures d'écran ci-dessous.

L'application est fluide, stable et respecte l'architecture MVC.

9. Conclusion

Ce projet a permis de comprendre la gestion des interfaces graphiques en Java, les interactions utilisateur et l'utilisation des threads pour animer une application.

L'architecture MVC a facilité l'organisation du code et la séparation des responsabilités entre le modèle, la vue et le contrôleur. L'ajout d'une fonctionnalité de détection de collision a montré comment le projet peut être étendu de manière progressive.

Ce travail constitue une base solide pour des projets plus complexes intégrant de nouvelles fonctionnalités et des comportements plus avancés.